

FLOWERAAINE: E-COMMERCE & AUTOMATED ORDER MANAGEMENT SYSTEM

1. Introduction

Many small businesses and independent creators operate entirely on social media platforms like Instagram. While these platforms are excellent for building an audience and marketing products, they lack the underlying software tools needed to handle complex retail operations. This is especially true for customized products that require specific buyer inputs. Currently, the owner of Floweraaine handles all custom gift orders, photo uploads, and payment verifications manually through direct messages. This informal process takes up hours of administrative time, increases the risk of losing important customer reference photos, and makes it incredibly difficult to enforce strict store policies like no cash-on-delivery.

Floweraaine is a custom e-commerce web application designed to completely automate this workflow. The brand specializes in handmade, customized gifts and décor, specifically 3D car frames, personalized hampers, and scented flower bouquets. By moving customers out of the Instagram inbox and onto a dedicated platform, this system provides a seamless way for buyers to customize their gifts and upload high-resolution reference photos. Simultaneously, it automatically links those uploaded photos to a secure database and equips the business owner with a centralized administrative dashboard to track every order from initial payment verification to final delivery.

2. Objectives

- To transition the business away from manual social media chats and provide a dedicated online storefront.
- To build a customization page where customers can type in text and upload reference photos directly for personalized items.
- To automate media storage by securely linking every uploaded photo to a specific Order ID in the database, ensuring no files are lost.
- To create a secure admin dashboard where the business owner can easily verify UPI payment screenshots and update the shipping status of orders.

3. Stakeholders

Stakeholders represent the key entities interacting with, operating, and maintaining the system.

1. Client (Business Owner)

- Role / Designation: Brand Owner and Primary Operating Authority.
- Operational Context: Manages daily workshop production, catalog item updates, order verification, and parcel dispatch.
- System Value: Eliminates hours of manual administrative message management, prevents lost reference files, and provides clear daily operational visibility.

2. Customers (Retail Users)

- Role / Designation: End Consumers and Gift Buyers.
- Target Audience: Car enthusiasts (seeking custom 3D vehicle frames), couples, families, and individuals purchasing unique handmade gifts and personalized bouquets.
- Actions Performed:
 - Access the public web application via desktop or mobile web browsers.
 - Browse the categorized product catalog featuring 3D Car Frames, Customized Gift Hampers, and Scented Flower Bouquets.
 - Utilize input fields and file upload pickers to attach personalized text and high-resolution reference photos.
 - Review shopping cart items and explicitly agree to store terms and fulfillment conditions.
 - Scan generated UPI payment QR codes, submit transaction reference screenshots, and monitor order status updates.

3. Store Administrator

- Role / Designation: Backend Operations Manager (Business Owner).
- Actions Performed:
 - Authenticate securely into the administrative dashboard.
 - View real-time incoming order queues organized by payment and production status.
 - Inspect uploaded customer UPI payment screenshots against order totals.

- Access and download high-resolution customer reference photos linked directly to specific Order IDs.
- Update order lifecycle states to trigger automated status updates for customers.

4. Technical User / System Administrator

- Role / Designation: Software Developer (Omith Thilakan).
- Actions Performed:
 - Maintain MongoDB database schemas and database index integrity.
 - Monitor REST API endpoints, Node.js server performance, and Cloudinary storage integrations.
 - Manage application deployment configurations, environment variables, and web security protocols.

4. Expected Modules and Sub-modules

Module 1: Storefront and Customization Engine

This module handles the public-facing retail experience, providing responsive navigation and real-time custom product configuration.

- Product Catalog Sub-module: Displays available handmade items divided into dedicated categories (3D Car Frames, Hampers, Bouquets). Features dynamic filtering, item detail modals, base pricing, and availability flags.
- Custom Gifting Sub-module: Provides conditional input fields based on product requirements. For instance, selecting a 3D Car Frame renders fields for custom vehicle registration numbers, text lines, and a dedicated image uploader for vehicle reference photos.
- Interactive Shopping Cart Sub-module: Manages local item state, allowing users to modify quantities, view itemized price totals, inspect preview thumbnails of uploaded assets, and proceed to checkout.

Module 2: Checkout, Policy & Payment Handling

This module manages transaction routing, policy compliance validation, and offline-to-online payment verification.

- Policy Enforcement Sub-module: Implements interactive validation checkpoints at checkout. Users must explicitly toggle mandatory checkboxes acknowledging store rules

(No COD, Opening Video required for claims, custom item processing times) before order submission endpoints are unlocked.

- Direct UPI QR Sub-module: Generates dynamic, client-side SVG QR codes formatted with the owner's merchant UPI address (upi://pay) and the exact order total, enabling instant scanning via any UPI app (GPay, PhonePe, Paytm).
- Payment Screenshot Verification Sub-module: Provides a file submission interface where buyers attach their payment confirmation screenshot and transaction UTR/Ref number, linking payment proof directly to the order record.

Module 3: Admin Order Management & Analytics Dashboard

This module serves as the command center for the business owner to fulfill orders and track operational metrics.

- Order Lifecycle State Machine Sub-module: Displays an interactive dashboard organizing orders into status columns (Pending Payment, Verification Required, In Production, Dispatched, Delivered). Admins can update order states with a single click.
- Secure Media Vault Sub-module: Automatically aggregates all customer-uploaded reference photos into an organized administrative interface. Photos are grouped by Order ID with single-click download capabilities for workshop preparation.
- Basic Sales Analytics Sub-module: Summarizes total revenue, daily order volumes, popular product categories, and pending payment queue statistics using clean data visualization components.

5. Hardware and Software Requirements

Hardware Requirements

- Minimum Processor: Dual-Core Intel Core i3 / AMD Ryzen 3 (2.0 GHz or higher)
- Primary Memory (RAM): 4 GB Minimum (8 GB Recommended)
- Secondary Storage: 256 GB Solid State Drive (SSD)
- System Architecture: 64-bit x86 Operating Architecture

Software Requirements

Operating System & Runtime

- Operating System: Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
- Programming Languages: JavaScript (ECMAScript 2022+ / ES6+), JSX

- Runtime Environment: Node.js (v18.0.0 or higher)

Frontend Framework & Client Libraries

- React.js 18: Component-based UI framework for building dynamic single-page application interfaces.
- Vite: Next-generation frontend build tool and local development server providing instant Hot Module Replacement (HMR).
- Tailwind CSS (v4): Utility-first CSS framework for custom responsive styling.
- Lucide React: Icon library for lightweight vector interface icons.
- Recharts: Composable charting library for rendering administrative sales analytics graphs.
- qrcode.react: SVG QR code generation library for dynamic UPI payment rendering.
- Axios: Promise-based HTTP client for executing REST API requests to the backend server.

Backend Runtime, Server Framework & Security

- Express.js: Web application framework for Node.js powering RESTful API endpoints and middleware routing.
- JSON Web Token (JWT): Secure, stateless authentication standard for managing administrative session persistence.
- BcryptJS: Hashing library for encrypting administrative passwords prior to database storage.
- Multer: Middleware for handling multipart/form-data, used for processing image uploads (reference photos and payment screenshots).

Database & Cloud Infrastructure

- MongoDB Community Server (v6.0+): Document-oriented NoSQL database used for storing unstructured product variants and order documents.
- Mongoose ORM: Object Data Modeling (ODM) library providing schema-based models for application data.
- Cloudinary SDK: Cloud-based image management service for storing, optimizing, and delivering user-uploaded photos.
- MongoDB Compass: Graphical User Interface (GUI) tool for local database administration and query testing.

Network & Deployment Protocols

- Network Environment: Localhost (Development) / Web Cloud Server (Production)
- Protocols: HTTP/1.1, HTTPS, REST API over TCP/IP
- Port Allocation:
 - Frontend Client: Port 5173
 - Backend REST API Server: Port 5000
 - MongoDB Instance: Port 27017

6. Similar Software Study and Background Study

Reference Application: Shopify

- Title: Shopify
- Type: Commercial Cloud-based E-Commerce Platform.
- Hardware and Software Requirements: Any modern operating system with a standard web browser and active internet connectivity.
- Core Modules: Hosted Storefront, Admin Inventory Manager, Order Fulfillment Center, Standard Checkout Engine.
- Salient Features:
 - Offers a drag-and-drop store builder with pre-designed themes.
 - Provides inventory management, automated tax calculations, and shipping label generation.
 - Integrates with major third-party payment gateways (Credit Cards, PayPal).
- Limitations:
 - Lack of Native Media Uploads: Shopify's core product schema is designed for standard, off-the-shelf retail (e.g., clothing, electronics). Allowing customers to upload custom reference images (such as car photos or personal portraits) requires purchasing and configuring third-party app subscriptions, increasing monthly operational costs.
 - Incompatibility with Informal Local Payments: Shopify relies on structured automated payment gateways. It does not provide native workflows for scanning direct merchant UPI QR codes and uploading payment receipt screenshots for

manual verification—a preferred payment method for Indian micro-businesses to avoid gateway transaction fees.

- High Financial Overhead: Shopify charges recurring monthly subscription fees in addition to transaction fees on sales. For a local, micro-scale custom gifting business, these platform fees directly reduce profit margins.
- Remark: While Shopify is an industry-standard platform for generic retail, its rigid structure and subscription costs make it ill-suited for a localized custom gifting micro-business. Floweraaine provides a specialized software solution tailored directly to the client's operational needs. By engineering a native media upload pipeline, integrating direct UPI payment screenshot verification, and enforcing local store policies at checkout, Floweraaine offers a purpose-built system without recurring platform fees.